

fox-webusb v0.0.0a1

Detailed compatibility matrix vs. Chrome / Chromium WebUSB

English technical report • 17 September 2026

Scope: page-facing WebUSB API, security model, transfer behavior, iframe/origin handling, hotplug, errors, and architecture. The comparison uses the supplied fox-webusb v0.0.0a1 source tree as the implementation authority and current public WebUSB/Chromium documentation as the browser reference.

Rating legend: Full = materially aligned at the WebUSB API/security-contract level; Partial = API exists but an observable or environment-dependent difference remains; Extra = fox-webusb behavior with no standard Chrome WebUSB equivalent; No = deliberately absent; N/A = architectural comparison rather than a pass/fail API feature; Not proven = insufficient evidence.

1. Executive summary

This document is a strict compatibility matrix for **fox-webusb v0.0.0a1** against the WebUSB behavior exposed by Chrome/Chromium. It is intentionally more conservative than a marketing compatibility list: a feature is marked **Full** only when the supplied fox-webusb source clearly implements the corresponding surface and the semantics are compatible enough to be described as aligned. Where Chrome has browser-internal behavior that cannot be reproduced by a Firefox extension + Native Messaging architecture, the difference is recorded explicitly.

Important scope: “Chrome WebUSB” here means the WebUSB API/specification plus Chromium security/implementation behavior used as the reference. It does not mean a byte-for-byte copy of a particular Chrome release. Chromium is actively evolving its WebUSB security hardening, so exact internal behavior can differ by browser version.

Overall assessment: fox-webusb has a broad WebUSB-shaped API surface, including the major USB object types, device selection, permissions, transfers, hotplug events, filters, protected-interface checks, and a security-key blocklist. The largest compatibility gaps are architectural rather than cosmetic: Firefox extension transport replaces browser-native USB plumbing; some Chrome-specific privileged mechanisms are intentionally absent; isochronous transfers depend on the available pyusb/libusb backend; and fox-webusb adds several extension-specific capabilities that Chrome WebUSB does not expose.

2. API object model

Area	Chrome / Chromium reference	fox-webusb v0.0.0a1	Status	Strict notes
navigator.usb	Browser-provided USB object in supported secure contexts.	Injected by the MAIN-world page polyfill when no existing navigator.usb is present and the context/policy checks permit installation.	Full	Same web-facing entry point; implementation is extension-provided rather than browser-native.
USB interface	EventTarget-like USB object with getDevices(), requestDevice(), connect and disconnect.	Shared webusb_core.js implements USB surface; page_polyfill.js exposes it to page code.	Full	Core object model is deliberately shared between the page-facing and Firefox-style surfaces.
USBDevice	Device metadata, configurations, opened state, and device control/transfer methods.	Implemented with vendor/product IDs, strings, USB/device version fields, configurations, opened state, claim tracking and transfer methods.	Full	Metadata is built from real USB descriptors where possible.
USBConfiguration	configurationValue, configurationName, interfaces.	Implemented.	Full	Configuration tree is generated from device descriptors.
USBInterface	interfaceNumber, alternates, claimed.	Implemented, including claimed state.	Full	Claim state is tracked per device handle.
USBAlternateInterface	alternateSetting, class/subclass/protocol, name, endpoints.	Implemented.	Full	Descriptor-derived values are exposed.
USBEndpoint	endpointNumber, direction, type, packetSize.	Implemented.	Full	Descriptor-derived endpoint tree.
USBInTransferResult	status, data.	Implemented.	Full	Status/error mapping is performed across the native boundary.
USBOutTransferResult	status, bytesWritten.	Implemented.	Full	Matches the WebUSB result shape.
Isochronous result/packet types	USBIsochronousIn/OutTransferResult and packet objects.	Classes and methods are present.	Partial	API surface exists, but actual isochronous backend support is environment-dependent and explicitly not guaranteed.
USBConnectionEvent	connect/disconnect event type for USB.	Implemented and dispatched through the extension/background/native-host path.	Full	Event delivery is restricted to origins that have granted the relevant device.

3. USBDevice methods

USBDevice method	Chrome reference	fox-webusb v0.0.0a1	Status	Notes
open()	Starts a device session.	Native host opens the selected USB device and assigns a per-handle ID.	Full	Per-origin handle ownership is enforced.
close()	Closes the device session.	Implemented; handle is closed and local state cleared.	Full	Native handle lifecycle is explicit.
forget()	Removes the device permission/pairing for the origin.	Implemented through the permission store.	Full	Persistent grant state is origin-scoped.
selectConfiguration()	Selects a configuration.	Implemented through pyusb/libusb.	Full	Backend/OS behavior can still constrain hardware.
claimInterface()	Claims an interface; protected classes are blocked by WebUSB security rules.	Implemented with protected-interface checks and per-handle/per-interface tracking.	Full	Security check is intentionally enforced in the native host.
releaseInterface()	Releases a claimed interface.	Implemented.	Full	Ownership is checked against the origin/handle.
selectAlternateInterface()	Selects an alternate setting.	Implemented; alternate setting is tracked per interface.	Full	Security hardening explicitly addresses alternate-setting confusion.
controlTransferIn()	USB control IN transfer with WebUSB setup object.	Implemented with requestType/recipient validation and native-side protected-class checks.	Full	Chromium has continued strengthening protected control-transfer checks; exact release-level behavior should be regression-tested.
controlTransferOut()	USB control OUT transfer.	Implemented with validation.	Full	Native side validates request fields and transfer sizes.
transferIn()	Bulk/interrupt IN transfer.	Implemented.	Full	Actual compatibility depends on USB backend/device.
transferOut()	Bulk/interrupt OUT transfer.	Implemented.	Full	Actual compatibility depends on USB backend/device.
clearHalt()	Clears endpoint halt.	Implemented.	Full	Native backend operation.
reset()	Resets device and cancels active operations.	Implemented through native host.	Full	Exact cancellation timing is backend/OS dependent.
isochronousTransferIn()	Isochronous IN transfer.	API method exists; native path validates packet lengths and calls backend support when available.	Partial	Not guaranteed across pyusb/libusb environments; source explicitly treats support as conditional.
isochronousTransferOut()	Isochronous OUT transfer.	API method exists; backend support is conditional.	Partial	Physical-hardware verification remains a limitation.

4. Device selection and filters

Filter / selection feature	Chrome / WebUSB reference	fox-webusb	Status	Strict notes
vendorId	Supported filter field.	Supported and matched in native host.	Full	Required relationship checks are also validated.
productId	Supported; paired with vendorId.	Supported; malformed productId-without-vendorId is rejected.	Full	JS and native validation cooperate.
classCode	Supported filter field.	Supported through USB interface/device matching logic.	Full	Descriptor-derived matching.
subclassCode	Supported; depends on classCode.	Supported; structural dependency is checked.	Full	Invalid hierarchy is rejected.
protocolCode	Supported; depends on subclassCode.	Supported; structural dependency is checked.	Full	Invalid hierarchy is rejected.
serialNumber	Supported filter field.	Supported where the device exposes a serial number.	Full	Unavailable serial strings naturally cannot match.
filters array validation	requestDevice() requires a valid filter structure.	Early TypeError validation plus deeper native matching.	Full	This is a compatibility/security validation layer, not merely a convenience check.
exclusionFilters	Supported in current WebUSB API/specification.	Accepted and validated by the shared core; native candidate filtering is used.	Full	Exact edge-case parity should be tested against the target Chrome release.
User selection chooser	Browser-owned chooser UI.	Native Tkinter chooser process/thread path.	Partial	Semantics are analogous, but UI and browser integration are inherently different.

5. Security, policy and permissions

Security / policy feature	Chrome / Chromium	fox-webusb v0.0.0a1	Status	Difference / assessment
Secure context requirement	WebUSB is restricted to secure contexts; localhost/file handling follows browser rules.	Polyfill checks window.isSecureContext and refuses installation when false.	Full	The extension does not invent a general insecure-context exception.
Transient user activation for requestDevice()	Required for page requestDevice().	Page surface checks userActivation; native dispatch carries gesture state.	Full	Background Firefox-style surface has different interaction constraints; see dedicated row below.
Permissions Policy: usb	usb directive controls availability; default allowlist is self.	page polyfill checks document.permissionsPolicy.allowsFeature("usb") and can withhold navigator.usb.	Full	Cross-origin iframe behavior depends on Firefox exposing the relevant Permissions Policy API; the implementation includes a conservative compatibility fallback when that API is absent.
Origin attribution	Browser owns the origin/frame security context.	Background validates sender.url and does not trust a page-supplied origin string.	Full	This is one of the most important architectural substitutions.
Frame-aware routing	Browser-native frame security and permission model.	Content script is injected into all frames; extension tracks tab/frame registrations and origin closure.	Partial	Equivalent security intent, but not literally Chromium's internal frame security machinery.
Protected interface classes	WebUSB protected interface classes are blocked from claim/control paths.	Eight protected classes are explicitly blocked: Audio, HID, Mass Storage, Hub, Smart Card, Video, Audio/Video, Wireless Controller.	Full	Source states this list was ported from WebUSB/Chromium reference material.
Security-key device blocklist	Chromium maintains a vendor/product blocklist and other hardening.	A static vendor/product blocklist is included and checked in the native host.	Partial	fox-webusb snapshot is not a live Chromium blocklist; Chromium main can change independently.
Protected control-transfer hardening	Chromium has added/strengthened checks for control transfers targeting protected classes.	Native bridge validates control-transfer request type/recipient and protected-class conditions.	Partial	Intent is aligned; exact byte-for-byte parity with each Chrome build is not claimed.
usb-unrestricted	Chromium has an Isolated Web App / privileged mechanism to bypass some protected-class/blocklist restrictions under controlled conditions.	Intentionally not implemented.	No	README explicitly rejects this because the Firefox extension architecture does not provide the same security foundation.
Permission persistence	Browser stores grants/pairing for permitted devices/origins.	Settings store persists origin/device grants.	Full	Storage mechanism is project-specific rather than Chrome profile storage.
Per-handle origin lock	Browser process owns device session and permission boundaries.	Every native handle is associated with an origin; cross-origin handle use is rejected.	Full	Strong project-side invariant.
Open-handle cap	Browser implementation has resource management but exact public cap is implementation-specific.	64 open handles per origin is explicitly enforced.	Extra	Project-specific safety limit; not a WebUSB API guarantee.

6. Architecture and transport

Architecture / transport	Chrome	fox-webusb	Status	Why it matters
USB backend	Browser-native Chromium USB service / OS integration.	Python pyusb/libusb in a separate Native Messaging host.	N/A	Architectural difference; web API can still be compatible.
Page-to-native path	Internal browser IPC; hidden from web page.	MAIN-world polyfill → window.postMessage → content script → background → Native Messaging → Python host.	N/A	Extra attack surface and failure modes exist by construction.
Native Messaging framing	Not exposed as a WebUSB concern.	Firefox Native Messaging with 4-byte length-prefixed messages; host handles binary stdio requirements.	N/A	Transport-level constraint unique to fox-webusb.
Browser restart behavior	Browser manages internal USB service state.	Extension and native host lifecycle are separate and must reconnect.	Partial	API semantics are similar but operational lifecycle is different.
Device chooser ownership	Browser UI / permission prompt.	External Tkinter chooser controlled by native host.	Partial	Same user-consent role, different UI trust boundary.

7. fox-webusb extensions beyond Chrome WebUSB

fox-webusb-specific feature	Chrome WebUSB	Status	Assessment
Local Ed25519 attestation	No equivalent WebUSB page API.	Extra	Origin-scoped public key + challenge signature. Useful for identifying the local bridge, but it is not Chrome WebUSB compatibility.
window.__foxWebUSB.bridgeInfo()	No standard equivalent.	Extra	Debug/diagnostic extension API.
window.__foxWebUSB.listGrantedDevices()	No standard equivalent page API.	Extra	Introspection helper; must not be confused with navigator.usb.getDevices().
window.__foxWebUSB.explainTransferLimits()	No standard equivalent.	Extra	Documents the project's warning/safety ceiling policy.
Firefox-style privileged surface	Chrome extension service-worker WebUSB support has its own browser-controlled model.	Partial	fox-webusb uses a persistent Manifest V2 background page and its own dispatch path; conceptual similarity does not imply identical privilege semantics.
Host-side grant/revoke management	Browser UI/profile controls are not exposed as this exact API.	Extra	Project-specific management surface.

8. Error compatibility

Error / validation behavior	Chrome / WebUSB reference	fox-webusb v0.0.0a1	Status	Notes
NotFoundError	Used for no selected/available device in request flow.	Native host returns NotFoundError strings and JS maps them to DOMException.	Full	Explicitly tested for chooser cancellation.
InvalidStateError	Used for invalid device state, e.g. operations before open.	JS requireOpen() and native state checks produce InvalidStateError.	Full	Core error mapping is shared.
SecurityError	Used for security-policy/permission failures.	Native security checks return SecurityError and JS converts it.	Full	Exact message text may differ.
InvalidAccessError	Used for invalid USB access parameters/state.	Mapped from native errors.	Full	Semantics are aligned at API level.
IndexSizeError	Used for invalid sizes/indices where applicable.	Native and JS validation can emit IndexSizeError.	Full	Exact edge-case parity needs browser-version regression tests.
TypeError	Used for invalid JS argument structures in spec-defined cases.	Explicitly preserved as built-in TypeError for filter validation and similar cases.	Full	Good compatibility detail: not every error is collapsed into DOMException.
Unknown native errors	Browser maps implementation failures into browser-defined DOMException/error behavior.	Unknown error prefixes are conservatively converted to NetworkError.	Partial	Observable error names/messages can differ from Chrome.

9. Resource and transfer limits

Resource / transfer constraint	Chrome reference	fox-webusb v0.0.0a1	Status	Strict assessment
Large transfer policy	Chromium has implementation/security limits and behavior that can vary by transfer path/version.	Project documents transfer limits and applies host-side validation; source also preserves WebUSB-compatible result/error semantics.	Partial	Do not assume exact Chrome byte-for-byte behavior.
Native Messaging message size	Not a Chrome WebUSB constraint.	Firefox Native Messaging imposes transport-level message constraints; host includes binary stdio handling and chunked stdout writing.	Extra	Large WebUSB payloads may be constrained by the Firefox transport even when USB itself can handle them.
Isochronous packet validation	WebUSB validates packet lengths according to API rules and browser implementation.	Native host validates total packet length and backend support.	Partial	Hardware/backend support is the limiting factor.
Open handle count	Not a public WebUSB API guarantee.	64 handles per origin.	Extra	Explicit safety cap.

10. Detailed compatibility verdict

Area	Verdict	Reason
Web-facing API shape	High compatibility	The supplied implementation intentionally reuses a single WebUSB core and exposes navigator.usb with the expected USB/USBDevice/configuration/interface/endpoint/result objects and the major device/transfer methods.
Device selection	High compatibility with UI differences	requestDevice(), filters, exclusion filters, user gesture checks, and origin-scoped grants are present. The chooser itself is native-host/Tkinter rather than browser-owned.
Permissions and origins	High security alignment; different plumbing	The native host binds handles to origins, validates extension sender URLs, persists grants, and routes hotplug events only to granted origins.
Protected devices	Strong alignment, but blocklist snapshot risk	Protected interface classes are explicitly blocked and a static security-key blocklist is included. The blocklist cannot automatically stay synchronized with Chromium main.
Iframe / Permissions Policy	High, with Firefox-dependent edge behavior	The polyfill checks the Permissions Policy usb feature and content scripts run in all frames. This reproduces the intended policy gate, not Chromium's internal implementation.
Transfers	High for control/bulk/interrupt; partial for isochronous	Core methods exist and native validation is substantial. Isochronous operation depends on backend support and remains experimentally constrained.
Errors	High API-level compatibility	Known WebUSB-style exception names are mapped deliberately, including preserving TypeError where appropriate. Unknown native failures are normalized to NetworkError, so exact error strings/names can differ.
Chrome-specific privileged features	Not equivalent	usb-unrestricted is intentionally absent. The Firefox-style surface is a project-specific extension surface, not a claim of identical Chrome extension privileges.
Overall	Broad API compatibility, not browser equivalence	fox-webusb v0.0.0a1 should be described as a WebUSB-compatible Firefox implementation with Chromium-inspired security hardening—not as “Chrome WebUSB running inside Firefox.”

11. Test and evidence notes

The v0.0.0a1 source tree contains JavaScript tests for the page polyfill, Python tests for the bridge/protocol/hardening/error/settings layers, and end-to-end native-host subprocess tests. The compatibility table does not treat the existence of a test as proof of Chrome parity; it treats it as evidence that the fox-webusb behavior is intentional and exercised.

The source explicitly documents several v0.0.0a1 security changes: Permissions Policy handling, user-activation checks, alternate-setting hardening, device-string sanitization, open-handle limits, and local attestation. It also explicitly documents the deliberate non-implementation of Chrome’s usb-unrestricted mechanism.

A particularly important distinction is the static Chromium security-key blocklist. The fox-webusb source identifies its list as derived from Chromium’s blocklist at a point in time. Chromium main is live code and has continued changing WebUSB control-transfer protections in 2026; therefore “Full” blocklist compatibility would be too strong. The matrix correctly marks it Partial.

12. Reference sources

WICG WebUSB specification: <https://wicg.github.io/webusb/>

MDN WebUSB API overview: https://developer.mozilla.org/en-US/docs/Web/API/WebUSB_API

MDN USB.requestDevice(): <https://developer.mozilla.org/en-US/docs/Web/API/USB/requestDevice>

MDN USB.getDevices(): <https://developer.mozilla.org/en-US/docs/Web/API/USB/getDevices>

MDN USBDevice: <https://developer.mozilla.org/en-US/docs/Web/API/USBDevice>

MDN Permissions-Policy usb directive: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Permissions-Policy/usb>

Chromium WebUSB security feature flags: https://chromium.googlesource.com/chromium/src/+/_master/services/device/public/cpp/device_features.cc

Chromium 2026 protected control-transfer hardening change: https://chromium.googlesource.com/chromium/src/+/_b334f9d5a8d0a54b39b0f17aa2149cdb3fd7feab

Chromium 2026 device-level protected-class hardening change: https://chromium.googlesource.com/chromium/src/+/_b334f9d5a8d0a54b39b0f17aa2149cdb3fd7feab

fox-webusb v0.0.0a1 supplied source tree: extension/webusb_core.js, extension/page_polyfill.js, native-host/src/fox_webusb_host/bridge.py, hardening.py, protocol.py, tests/.

Document conclusion: The correct technical description of fox-webusb v0.0.0a1 is “a Firefox extension + Native Messaging WebUSB-compatible implementation with Chromium-inspired security hardening.” It is not technically accurate to describe it as a full Chrome WebUSB clone, because the browser/USB architecture, chooser, transport, privileged features, blocklist update mechanism, and some backend-dependent transfer semantics differ.